

Fourier Series & CFT - Online 3 Practice Guide

Full version of the previous chat reply - expanded instead of summarized. It includes the complete reasoning, all 34 practice questions, the three mock exams, the probability ranking, and the formula cheat sheets.

After looking through the material, I think your expectation is reasonable: this Online 3 is very likely to be a **small change to Offline 2 / reuse-your-previous-code** exam rather than a completely new implementation from scratch.

Important exam strategy: Your Offline 2 gave you reusable building blocks. The key skill is not memorizing every line of the old assignment, but recognizing which routine can be reused and what tiny part has to change.

What was compared

The analysis was based on the current-year Offline 2, previous CSE 220 Online/Offline material, especially the 22-batch Online 3 CFT papers, the 22-batch CFT Offline, and the three theory lectures.

The current Offline 2 gives you exactly these reusable building blocks:

```
FourierEpicycles
-> calculate_cn()
-> calculate_all_coefficients()
-> approximate()

CFT2D
-> compute_cft()
-> plot_magnitude()

InverseCFT2D
-> reconstruct()
```

The current Offline 2 asks for complex Fourier coefficients over negative and positive harmonics and a separable 2D CFT implementation. That makes the overlap with an online extension particularly strong.

Also, previous Online 3 papers were more focused: derivative property, time shifting of a Gaussian, frequency shifting plus time scaling, and numerical verification. So a 30-40 minute exam is much more likely to give you a template and ask you to **modify or extend one known piece** than make you recreate all of Offline 2.

What I think you should practice

The questions below are divided into:

- A. Very high probability
- B. Medium probability
- C. 2D-CFT questions
- D. Evil questions I would still prepare for because Offline 2 was hard

A. VERY HIGH-PROBABILITY QUESTIONS

Fourier Series side

1. Compute one complex Fourier coefficient

You are given a sampled signal, for example: $t = \text{np.linspace}(0, 2\pi, 2000)$ $x = \sin(t) + 2\cos(2t)$ Implement: `calculate_cn(n)` using $c_n = (1/T) \int x(t) \exp(-j n \omega_0 t) dt$. No FFT. **Expected reuse:** your `calculate_cn()` from Offline 2.

2. Compute all coefficients from -N to N

Given `n_harmonics = 20`, calculate: `c[-20], ..., c[0], ..., c[20]` and store them in the structure expected by the template. **Trap:** don't calculate only `0...N`. Your Offline 2 explicitly works with negative and positive harmonics.

3. Reconstruct the signal from the coefficients

Given `coeffs` and `t_query`, implement: `approximate(t)` using: $\hat{f}(t) = \sum_{n=-N}^N c_n \exp(j n \omega_0 t)$. The examiner might give you `calculate_cn()` already and ask only for `approximate()`.

4. Reconstruct only selected harmonics

Given coefficients for: $n = -20, \dots, 20$ reconstruct using only: $n = -5, \dots, 5$ Then plot original vs approximation. This is basically your existing epicycle reconstruction with a change in which coefficients participate.

5. Find the dominant harmonics

Given coeffs, find the K coefficients having largest magnitude: $\text{abs}(c_n)$ and reconstruct the signal using only those. This is a natural extension of your epicycle implementation.

6. Plot magnitude and phase spectrum

Given a coefficient dictionary: $\{-2: \dots, -1: \dots, 0: \dots, 1: \dots, 2: \dots\}$ generate: $|c_n|$ $\arg(c_n)$ against n . Lecture 5 explicitly introduces magnitude and phase spectra and the real-signal property $c_{-k} = \text{conjugate}(c_k)$.

7. Time-shift an already-known Fourier series

Suppose: $x(t) \leftrightarrow c_k$ and: $y(t) = x(t - 2)$ Find the new coefficients. Use: $c'_k = c_k \exp(-j k \omega_0 t_0)$. Coding version: given the old coefficient array, generate the shifted coefficient array without recalculating the integral. This is an extremely exam-friendly question.

8. Time reversal

Given: $y(t) = x(-t)$ and the Fourier coefficients of x , calculate the coefficients of y . You should know: $c'_k = c_{-k}$.

9. Time scaling

Given: $y(t) = x(3t)$ what happens to: - coefficients? - fundamental frequency? - period? The coefficient sequence stays the same while the fundamental frequency becomes $3 \omega_0$. This is a very exam-friendly coding question.

10. Combine shift + reversal + scaling

Given: $y(t) = x(4 - 3t)$ and the coefficients of x , find the new coefficients. This is almost literally the lecture example. Practice applying shift $\rightarrow$ reverse $\rightarrow$ scale.

CFT side

11. Implement a 1D CFT

Given: $x(t)$ frequencies compute: $X(f) = \int x(t) \exp(-j 2 \pi f t) dt$ using trapezoidal integration. No FFT. This is basically the core of previous CFT Online questions.

12. Verify time-shifting property numerically

Given: $x(t)$ $y(t) = x(t - t_0)$ calculate: $X = \text{CFT}(x)$ $Y = \text{CFT}(y)$ and compare: $|X(f)|$ versus $|Y(f)|$ and $\angle(Y(f))$ versus $\angle(X(f)) - 2 \pi f t_0$. Then calculate MSE. This is very close to the previous Online 3 numerical-verification style.

13. Verify differentiation property

Given: $y = \text{derivative of } x$ verify: $Y(f) = j 2 \pi f X(f)$. This is especially plausible because previous Online 3 used derivative-property verification numerically.

14. Verify frequency-shift / modulation property

Given: $y(t) = x(t) \exp(j 2 \pi f_0 t)$ verify: $Y(f) = X(f - f_0)$.

15. Verify cosine modulation

Given: $y(t) = x(t) \cos(2 \pi f_0 t)$ verify: $Y(f) = 1/2 X(f - f_0) + 1/2 X(f + f_0)$.

16. Time scaling property

Given: $y(t) = x(at)$ verify: $Y(f) = (1/|a|) X(f/a)$. This becomes especially likely because one previous Online 3 combined time scaling with frequency shifting.

B. MEDIUM-PROBABILITY QUESTIONS

17. Frequency shift + time scaling together

Given: $y(t) = x(at) * \exp(j 2 \pi f_0 t)$ verify: $Y(f) = (1/|a|) X((f - f_0)/a)$. This is practically the previous combined-property online style. Expect magnitude/phase comparison and possibly MSE.

18. Shift + scale + derivative

Given: $y(t) = d/dt [x(a \cdot t - t_0)]$ derive the expected frequency-domain expression and verify numerically. This is nasty but manageable if you know the individual properties.

19. Generate a Fourier series from known components

Given: $x(t) = 2 + 3 \cos(\omega_0 t) - 4 \sin(2 \omega_0 t)$ construct its c_k directly without numerical integration. This uses the lecture idea that Fourier components can sometimes be read directly from the signal.

20. Verify a Fourier-series property numerically

Start from: $x(t)$ calculate its coefficients. Then generate: $y(t) = x(t - t_0)$ using the time-domain signal. Separately obtain predicted coefficients using: $c_{\text{shifted}} = c \cdot \exp(\dots)$ Compare the two coefficient sets. This tests understanding instead of blind formula copying.

21. Differentiate Fourier-series coefficients

Given c_k , construct the coefficients of: $y(t) = dx/dt$ using: $d_k = j k \omega_0 c_k$. Lecture 6 gives this transformation directly.

22. Integrate a periodic signal

Given Fourier coefficients of $x(t)$, calculate the coefficients of its integral. For $k \neq 0$: $d_k = c_k / (j k \omega_0)$ But the DC term must be handled separately. Lecture 6 explicitly warns about this.

23. Multiply two periodic signals

Given: $x(t) \leftrightarrow a_k$ $y(t) \leftrightarrow b_k$ calculate coefficients of: $z(t) = x(t)y(t)$ using convolution: $c_k = \sum_p a_p b_{(k-p)}$. This was taught explicitly, but I rank it below shift/scale/differentiation.

C. 2D-CFT QUESTIONS

24. Implement only compute_cft()

The examiner gives: `class CFT2D: ... def compute_cft(self): pass` Fill it using the Offline 2 separable strategy: $\cos(u x + v y)$ $\sin(u x + v y) \rightarrow$ angle addition $\rightarrow$ integrate over $x \rightarrow$ integrate over y . The current Offline 2 explicitly requires the separable $O(N^3)$ -type strategy instead of brute-force 4D integration. **This is one of the most important questions to practice.**

25. Implement only the inverse CFT

Given: real $u \ v \ x \ y$ implement: `reconstruct()` using the separable inverse transform. This is a likely "you already solved Offline 2, now do the inverse correctly" style question.

26. Change the high-pass filter into a low-pass filter

Current Offline 2 logic zeros the central low-frequency region for a high-pass edge detector. Now the exam asks: Modify it to make a low-pass filter. Instead, preserve the central region and zero frequencies outside the cutoff.

27. Band-pass filter

Given a 2D spectrum, keep only: $r_{\text{low}} < \text{distance} < r_{\text{high}}$ and zero everything else. Then reconstruct. This is a very natural extension of the current `FrequencyFilter`.

28. Remove a specific frequency band

For a 2D image, suppress: $\text{distance_from_origin} < \text{cutoff}$ or: $\text{distance_from_origin} > \text{cutoff}$ and compare reconstructed images. This tests whether you understand what the frequency-domain mask is actually doing.

29. Spectrum + reconstruction + error

Given: real, $\text{img} = \text{cft.compute_cft}()$ $\text{filtered_real}, \text{filtered_img} = \text{filter}(\dots)$ $\text{reconstructed} = \text{inverse.reconstruct}()$ calculate: $\text{MSE} = \text{mean}((\text{original} - \text{reconstructed})^2)$ and compare different cutoff values. This combines several Offline 2 components but remains feasible if the framework is given.

D. THE 'PROFESSOR IS TRYING TO FUCK US' QUESTIONS

30. Make FourierEpicycles handle arbitrary complex signals

Currently the input may be: $z = x + 1j*y$ The question could change the signal source to something generated mathematically instead of SVG. Then: `fs = FourierEpicycles(t, z, N)` should still work. This tests whether your implementation is actually generic.

31. Reconstruct a signal from coefficients but evaluate thousands of times at once

Given: `t = np.linspace(...)` the examiner expects: `approximate(t)` to work on an array containing thousands of time values, not one scalar at a time. This is explicitly part of the Offline 2 expectation and can burn a lot of time if broadcasting is unclear.

32. Debug a wrong CFT implementation

They give code using arrays such as: `ux = outer(...)` `vy = outer(...)` and ask why the spectrum is wrong. A likely issue is mixing spatial x/y axes with frequency u/v axes. Your Offline 2 warns about these distinctions.

33. Fix a broadcasting/shape error in separable CFT

For example, an expression involving: `self.I[:, None, :]` must be paired with correctly shaped x/u terms. The question may ask you to determine the shapes and make the multiplication work. This is exactly the sort of NumPy issue that appears in a timed coding exam.

34. Compute CFT using separability but with a different integration order

Instead of: x first -> y second they give skeleton code for: y first -> x second and expect the same final result. This tests whether you understand separability rather than memorizing dimensions.

Three MOCK EXAMS I WOULD ACTUALLY DO

MOCK 1 - 30 minutes - VERY LIKELY

Problem: You are given a class: `class FourierSeries: def __init__(self, t, signal, N): ...` The Fourier coefficients c_k are already calculated. Implement: `shifted_coefficients(t0)` `reversed_coefficients()` `reconstruct(t)` For: $y(t) = x(2 - t - t_0)$ derive the coefficient relationship and plot $x(t)$ and $y(t)$.

Skills tested: coefficient handling, time shift, reversal, reconstruction, NumPy vectorization. **Target:** 25-30 minutes.

MOCK 2 - 35 minutes - CFT PROPERTY STYLE

Given: $y(t) = x(10t) \exp(j 2 \pi 5 t)$ Compute: $X(f)$ $Y(f)$ and verify: $Y(f) = (1/10) X((f-5)/10)$. Plot: $|Y(f)|$ $(1/10) |X((f-5)/10)|$ and calculate MSE.

This is very close to the style of the previous CFT Online 3 combined-property questions.

MOCK 3 - 40 minutes - 'OFFLINE 2 FOLLOW-UP'

You are given your existing: `CFT2D` `FrequencyFilter` `InverseCFT2D` Modify the program so that instead of a high-pass edge detector it performs a **band-pass spatial-frequency filter**: $r_{\text{low}} < \sqrt{u^2 + v^2} < r_{\text{high}}$ Requirements: 1. Compute 2D CFT. 2. Construct the band-pass filtered spectrum. 3. Reconstruct image with inverse CFT. 4. Display: - original image - log magnitude spectrum - filtered image No FFT.

Why this matters: you do not need new mathematics. You need to understand and modify the Offline 2 pipeline.

My probability ranking for your actual Online 3

Topic	Probability
Modify/reuse Fourier coefficient calculation	★★★★★
Fourier-series reconstruction	★★★★★
Time shift	★★★★★
Time scaling	★★★★★
CFT numerical integration	★★★★★
CFT property verification	★★★★★
Derivative property	★★★★■
Frequency shifting / modulation	★★★★■
Shift + scale combined	★★★★■
2D CFT modification	★★★★■
Inverse CFT	★★★★■
Low/high/band-pass filtering	★★★★■
Magnitude/phase spectrum	★★★★■
Time reversal	★★★██
FS differentiation	★★★██
FS integration	★★★██
Fourier-coefficient convolution	★★███
Completely new 2D algorithm	★★███

The reason I put shift/scale/modulation/derivative so high is that Lecture 6/7 deliberately teach these as coefficient/spectrum transformations rather than requiring another full integral. That is exactly the sort of thing an online exam can turn into a 30-minute programming task.

What I would memorize before the exam

You do not need to memorize the entire Offline 2 code. Memorize the transformation rules and, more importantly, know where in your existing code to apply them.

Fourier Series

$x(t) \leftrightarrow c_k$
$x(t-t_0) \leftrightarrow c_k \exp(-j k \omega_0 t_0)$
$x(-t) \leftrightarrow c_{-k}$
$x(a t) \leftrightarrow c_k$, with the new fundamental frequency scaled
$d/dt x(t) \leftrightarrow j k \omega_0 c_k$
$\int x(t) dt \leftrightarrow c_k/(j k \omega_0)$, $k \neq 0$
$x(t)y(t) \leftrightarrow c_k = \sum_p a_p b_{(k-p)}$

CFT

$X(f) = \int x(t) \exp(-j 2 \pi f t) dt$
$x(t-t_0) \leftrightarrow X(f) \exp(-j 2 \pi f t_0)$
$x(t) \exp(j 2 \pi f_0 t) \leftrightarrow X(f-f_0)$
$x(t) \cos(2 \pi f_0 t) \leftrightarrow 1/2 X(f-f_0) + 1/2 X(f+f_0)$
$x(a t) \leftrightarrow (1/ a) X(f/a)$
$d/dt x(t) \leftrightarrow j 2 \pi f X(f)$

Final exam advice

Do not practice by rewriting the entire Offline 2 again and again. Your exam advantage is that you already fought through that monster. Practice taking the existing code and answering: **"What is the smallest part I need to change to satisfy this new requirement?"** That is much closer to the pattern shown by previous Online 3 papers.

The strongest next step is to sit down with the three mock exams above and do them under actual 30-40 minute timers. Do not look at your old code until you have identified which part of the existing structure you need to reuse.